



Tech & Teach

Digital Chip Design Program

RTL Design

Task #14

Farah Alyahya

Rana Alsubaihi

Joud Almshari

Tala Alrsheed

Hoor Alkhedairi

Horiah Alqufaydi

Sandra Alomar

1/8/2026

Controller & Datapath

Version 1.0

Task #1.1

1.1.1

Program Counter :

```
1 module ProgramCounter
2   #( parameter PROG_VALUE = 4 )
3   (
4     input logic clk,rstn,
5     output reg [($clog2(PROG_VALUE))-1:0] counter
6   );
7
8   always_ff @(posedge clk or negedge rstn) begin
9     if (!rstn)
10      counter <= 0;
11     else
12      counter <= counter + 1;
13   end
14 endmodule
```

Instruction Memory :

```
1 module InstructionMemory
2   #(
3     parameter IMEM_DEPTH = 16,
4     parameter FILE_NAME = "machine_code.mem"
5   )
6   (
7     input logic [$clog2(IMEM_DEPTH)-1:0] address,
8     output logic [7:0] instruction
9   );
10
11   logic [7:0] imem [IMEM_DEPTH-1:0];
12
13   initial begin
14     $readmemb(FILE_NAME, imem);
15   end
16
17   assign instruction = imem[address];
18 endmodule
```

ALU :

```
1 module ALU
2   #(parameter ALU_WIDTH = 4)
3   (
4     input logic [ALU_WIDTH-1:0] op1,op2,
5     input logic [1:0] opcode,
6     output logic [ALU_WIDTH-1:0] alu_result
7   );
8
9   always_comb begin
10     case(opcode)
11       2'b00: alu_result = op1 + op2;
12       2'b01: alu_result = op1 - op2;
13       2'b10: alu_result = op1 & op2;
14       2'b11: alu_result = op1 | op2;
15       default: alu_result = 'bx;
16     endcase
17   end
18 endmodule
```

Register File :

```
1 module RegisterFile
2   #(parameter REGF_WIDTH = 4)
3   (
4     input logic clk, rstn,
5     input logic [1:0] rs1,rs2,rd,
6     input logic [REGF_WIDTH-1:0] alu_result,
7     output logic [REGF_WIDTH-1:0] op1,op2
8   );
9
10   logic [REGF_WIDTH-1:0] x[0:3];
11
12   always@(posedge clk, negedge rstn) begin
13     if(!rstn) begin
14       for(int i = 0; i < 4; i++)
15         x[i] <= 0;
16     end
17     else begin
18       // storing alu result (rd)
19       case(rd)
20         2'b00: x[0] <= 0;
21         2'b01: x[1] <= alu_result;
```

```

22             2'b10: x[2] <= alu_result;
23             2'b11: x[3] <= alu_result;
24             default: begin
25                 x[1] <= x[1];
26                 x[2] <= x[2];
27                 x[3] <= x[3];
28             end
29         endcase
30     end
31 end
32
33 // rs1
34 always_comb begin
35     case(rs1)
36         2'b00: op1 = x[0];
37         2'b01: op1 = x[1];
38         2'b10: op1 = x[2];
39         2'b11: op1 = x[3];
40         default: op1 = 'b0;
41     endcase
42 end
43
44 // rs2
45 always_comb begin
46     case(rs2)
47         2'b00: op2 = x[0];
48         2'b01: op2 = x[1];
49         2'b10: op2 = x[2];
50         2'b11: op2 = x[3];
51         default: op2 = 'b0;
52     endcase
53 end
54 endmodule

```



ach

Datapath :

```
1 module Datapath
2   #(
3     parameter IMEM_DEPTH = 16,
4     parameter PROG_VALUE = 4,
5     parameter REGF_WIDTH = 4,
6     parameter ALU_WIDTH = 4,
7     parameter FILE_NAME = "machine_code.mem"
8   )(
9     input logic clk,
10    input logic rstn
11  );
12
13  logic [$clog2(IMEM_DEPTH)-1:0] address;
14  logic [7:0] instruction;
15  logic [ALU_WIDTH-1:0] op1,op2,alu_result;
16
17  ProgramCounter # (
18    PROG_VALUE
19  ) pc_ins (
20    .clk(clk),
21    .rstn(rstn),
22    .counter(address)
23  );
24
25  InstructionMemory # (
26    IMEM_DEPTH,
27    FILE_NAME
28  ) rom_ins (
29    .address(address),
30    .instruction(instruction)
31  );
32
33  RegisterFile # (
34    REGF_WIDTH
35  ) rf_ins (
36    .clk(clk),
37    .rstn(rstn),
38    .rs1(instruction[3:2]),
39    .rs2(instruction[5:4]),
40    .rd(instruction[7:6]),
41    .alu_result(alu_result),
42    .op1(op1),
43    .op2(op2)
44  );
45
46  ALU # (
47    ALU_WIDTH
48  ) alu_ins (
49    .op1(op1),
50    .op2(op2),
51    .opcode(instruction[1:0]),
52    .alu_result(alu_result)
53  );
54 endmodule
```

1.1.2

Control unit :

```
1 module Control (  
2     input  logic clk,rstn,  
3     input  logic [1:0] opcode,  
4     output logic reg_write  
5 );  
6  
7 always_comb begin  
8     if (!rstn) begin  
9         reg_write = 1'b0;  
10    end  
11    else begin  
12        case (opcode)  
13            2'b00: reg_write = 1'b1;  
14            2'b01: reg_write = 1'b1;  
15            2'b10: reg_write = 1'b1;  
16            2'b11: reg_write = 1'b1;  
17            default: reg_write = 1'b0;  
18        endcase  
19    end  
20 end  
21 endmodule
```

1.1.3

Top module :

```
1  module Top
2      #(
3          parameter IMEM_DEPTH = 16,
4          parameter PROG_VALUE = 4,
5          parameter REGF_WIDTH = 4,
6          parameter ALU_WIDTH = 4,
7          parameter FILE_NAME = "machine_code.mem"
8      ) (
9          input logic clk,
10         input logic rstn,
11         output logic [ALU_WIDTH-1:0] alu_result_top
12     );
13
14     logic [7:0] instruction;
15     logic reg_write;
16
17     Datapath #(
18         .IMEM_DEPTH(IMEM_DEPTH),
19         .PROG_VALUE(PROG_VALUE),
20         .REGF_WIDTH(REGF_WIDTH),
21         .ALU_WIDTH(ALU_WIDTH),
22         .FILE_NAME(FILE_NAME)
23     ) dp_ins (
24         .clk(clk),
25         .rstn(rstn),
26         .alu_result(alu_result_top),
27         .instruction_out(instruction)
28     );
29
30     Control ctrl_ins (
31         .clk(clk),
32         .rstn(rstn),
33         .opcode(instruction[1:0]),
34         .reg_write(reg_write)
35     );
36
37 endmodule
```

ch

1.1.4

Testbench :

```
1  `timescale 1ns / 1ps
2  module Top_tb();
3
4      logic clk,rstn;
5      logic [3:0] alu_result_top;
6
7      Top #(
8          .IMEM_DEPTH(16),
9          .PROG_VALUE(4),
10         .REGF_WIDTH(4),
11         .ALU_WIDTH(4),
12         .FILE_NAME("machine_code.mem")
13     ) dut (
14         .clk(clk),
15         .rstn(rstn),
16         .alu_result_top(alu_result_top)
17     );
18
19     always #5 clk = ~clk;
20
21     initial begin
22         clk = 0;
23         rstn = 0;
24
25         #10 rstn = 1;
26
27         #100;
28         $finish;
29     end
30 endmodule
```


Work distribution

Section	Farah	Joud	Tala	Horiah	Rana	Hoor	Sandra
Report	✓	✓	✓	✓	✓	✓	✓
Task 1.1	✓	✓	✓	✓	✓	✓	✓

Tech & Teach